

Migración a MySQL de Hostinger



Cambios Realizados

La aplicación ha sido migrada de **PostgreSQL (Supabase)** a **MySQL (Hostinger)** para aprovechar la base de datos incluida en el hosting.



Cambios en el Schema

1. Proveedor de Base de Datos

```
// Antes (PostgreSQL)
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// Después (MySQL)
datasource db {
  provider = "mysql"
  url      = env("DATABASE_URL")
  relationMode = "prisma"
}
```

2. Arrays → JSON

MySQL no soporta arrays nativos, todos los campos de tipo `String[]` han sido convertidos a `Json` :

```
// Antes
images      String[]
colors      String[]
styles      String[]
rooms       String[]

// Después
images      Json
colors      Json
styles      Json
rooms       Json
```

Importante: En tu código TypeScript, estos campos ahora se manejan como `any` o arrays en JSON:

```
// Ejemplo de uso
const product = await prisma.product.create({
  data: {
    name: "Wallpaper",
    images: ["url1", "url2"], // Prisma lo convierte automáticamente a JSON
    colors: ["red", "blue"],
    styles: ["modern", "classic"]
  }
});

// Al leer, TypeScript lo trata como 'any' pero puedes hacer cast
const images = product.images as string[];
```

3. Índices Agregados

Se han agregado índices para todas las foreign keys (requerido con `relationMode = "prisma"`):

```
model Account {
  // ...
  @@index([userId])
}

model Session {
  // ...
  @@index([userId])
}

// ... y así para todos los modelos con relaciones
```

Configurar Base de Datos MySQL en Hostinger

Paso 1: Crear Base de Datos

1. Inicia sesión en Hostinger Panel:

- <https://hpanel.hostinger.com/>

2. Navega a Bases de Datos:

- Dashboard → Bases de Datos → MySQL Databases

3. Crear nueva base de datos:

- Click en "Crear Nueva Base de Datos"
- Nombre: `u425976741_barrera` (o similar, Hostinger añade prefijo automáticamente)
- Usuario: Se genera automáticamente (ej: `u425976741_barrera`)
- Contraseña: Genera una contraseña segura

4. Guardar credenciales:

Host: `localhost` (en el mismo servidor)
 Puerto: `3306` (por defecto)
 Usuario: `u425976741_barrera`
 Contraseña: La que generaste
 Base de datos: `u425976741_barrera`

Paso 2: Construir DATABASE_URL

Formato:

```
mysql://[usuario]:[contraseña]@[host]:[puerto]/[base_datos]
```

Ejemplo:

```
mysql://u425976741_barrera:MiPassword123@localhost:3306/u425976741_barrera
```

⚠ Importante:

- NO uses espacios
- Escapa caracteres especiales en la contraseña si los hay
- El host normalmente es `localhost` en Hostinger



Ejecutar Migraciones

En Local (para desarrollo)

```
# 1. Actualizar .env con MySQL DATABASE_URL
DATABASE_URL="mysql://usuario:contraseña@localhost:3306/database"

# 2. Crear migración inicial
cd app
yarn prisma migrate dev --name init

# 3. Generar cliente Prisma
yarn prisma generate

# 4. Sembrar datos de prueba (opcional)
yarn prisma db seed
```

En Hostinger (producción)

```
# 1. Conectar por SSH
ssh u425976741@tu-dominio.com -p 65002

# 2. Ir al directorio de la app
cd ~/domains/tu-dominio/public_html/app

# 3. Asegurarte de que DATABASE_URL esté configurada
echo $DATABASE_URL
# Debe mostrar: mysql://...

# 4. Ejecutar migraciones
yarn prisma migrate deploy

# 5. Generar cliente Prisma
yarn prisma generate

# 6. Sembrar base de datos (solo primera vez)
yarn prisma db seed

# 7. Reiniciar aplicación
pm2 restart barrera-wallpaper
```

Verificar Conexión

Test de Conexión Manual

Crea un archivo `test-mysql.js` en el root del proyecto:

```

require('dotenv').config();
const { PrismaClient } = require('@prisma/client');

const prisma = new PrismaClient();

async function testConnection() {
  try {
    console.log('🔍 Testing MySQL connection...');
    console.log('DATABASE_URL:', process.env.DATABASE_URL?.replace(/:([^:]*@/,
':***@'));

    // Intentar una consulta simple
    const result = await prisma.$queryRaw`SELECT 1 as test`;
    console.log('✅ Connection successful!');
    console.log('Test query result:', result);

    // Contar usuarios
    const userCount = await prisma.user.count();
    console.log('✅ Found ${userCount} users in database`);

    await prisma.$disconnect();
    process.exit(0);
  } catch (error) {
    console.error('❌ Connection failed:');
    console.error(error);
    await prisma.$disconnect();
    process.exit(1);
  }
}

testConnection();

```

Ejecutar:

```
node test-mysql.js
```

**Diferencias Importantes PostgreSQL vs MySQL****Tipos de Datos**

Característica	PostgreSQL	MySQL
Arrays nativos	✅ Soportado	❌ No soportado (usar JSON)
JSON	✅ JSONB (binario)	✅ JSON (texto)
Text sin límite	✅ TEXT	✅ TEXT
Boolean	✅ BOOLEAN	⚠️ TINYINT(1)
UUID	✅ UUID	⚠️ CHAR(36)

Funcionalidades

Característica	PostgreSQL	MySQL
Foreign Keys	✓ Nativo	✓ Nativo (con InnoDB)
Transacciones	✓ Completo	✓ Completo
Full-text search	✓ Avanzado	⚠ Básico
Case sensitivity	✓ Sensible	⚠ Insensible por defecto



Solución de Problemas

Error: “Access denied for user”

Causa: Credenciales incorrectas o permisos insuficientes.

Solución:

1. Verifica usuario y contraseña en Hostinger Panel
2. Asegúrate de que el usuario tenga permisos en la base de datos
3. Verifica que la DATABASE_URL esté correctamente formateada

Error: “Unknown database”

Causa: La base de datos no existe.

Solución:

1. Crea la base de datos en Hostinger Panel
2. Verifica el nombre exacto de la base de datos
3. Usa el nombre con el prefijo correcto (ej: `u425976741_barrera`)

Error: “Can’t connect to MySQL server”

Causa: Host o puerto incorrectos.

Solución:

1. En Hostinger, el host suele ser `localhost`
2. Verifica que el puerto sea `3306`
3. Si estás fuera del servidor, MySQL puede no aceptar conexiones remotas

Error: “Prisma Client did not initialize yet”

Solución:

```
cd app
yarn prisma generate
yarn build
pm2 restart barrera-wallpaper
```

Tablas no se crean automáticamente

Solución:

```
cd app
yarn prisma migrate deploy
```



Notas de Migración de Datos

Si tienes datos en PostgreSQL que necesitas migrar a MySQL:

Opción 1: Export/Import Manual

```
# 1. Exportar datos de PostgreSQL (en Supabase)
# Usa la interfaz de Supabase o pg_dump

# 2. Convertir formato si es necesario
# Los arrays deben convertirse a JSON

# 3. Importar a MySQL
# Usa phpMyAdmin en Hostinger o MySQL CLI
```

Opción 2: Script de Migración

Crea un script Node.js que:

1. Se conecte a PostgreSQL (lectura)
2. Se conecte a MySQL (escritura)
3. Copie datos tabla por tabla
4. Convierta arrays a JSON automáticamente

Ejemplo básico:

```

const { PrismaClient: PrismaPostgres } = require('@prisma/client/postgres');
const { PrismaClient: PrismaMySQL } = require('@prisma/client');

const pgPrisma = new PrismaPostgres({
  datasources: { db: { url: POSTGRES_URL } }
});

const mysqlPrisma = new PrismaMySQL({
  datasources: { db: { url: MYSQL_URL } }
});

async function migrate() {
  // Migrar usuarios
  const users = await pgPrisma.user.findMany();
  for (const user of users) {
    await mysqlPrisma.user.create({ data: user });
  }

  // Migrar productos (convertir arrays a JSON)
  const products = await pgPrisma.product.findMany();
  for (const product of products) {
    await mysqlPrisma.product.create({
      data: {
        ...product,
        // Los arrays se convierten automáticamente
      }
    });
  }
}

migrate().catch(console.error);

```

✓ Checklist de Migración

Pre-Migración:

- [] Base de datos MySQL creada en Hostinger
- [] Credenciales anotadas (usuario, contraseña, host, puerto, database)
- [] DATABASE_URL construida correctamente
- [] Variable de entorno configurada en Hostinger

Migración:

- [] Schema Prisma actualizado a MySQL
- [] Arrays convertidos a JSON
- [] Índices agregados
- [] `yarn prisma generate` ejecutado
- [] `yarn prisma migrate deploy` ejecutado
- [] Datos sembrados (si es primera vez)

Post-Migración:

- [] Test de conexión exitoso
- [] Aplicación se inicia sin errores
- [] Login/registro funciona

- [] CRUD de productos funciona
- [] Órdenes se pueden crear
- [] Sin errores en logs de PM2

Resumen

Lo que cambió:

-  Provider: PostgreSQL → MySQL
-  Arrays → JSON
-  Agregados índices para foreign keys
-  DATABASE_URL: formato postgresql:// → mysql://

Lo que NO cambió:

-  Estructura de tablas (mismos modelos)
-  Relaciones entre modelos
-  API de Prisma en el código
-  Funcionalidad de la aplicación

Beneficios:

-  Sin costo adicional (incluido en Hostinger)
-  Mejor rendimiento (mismo servidor)
-  Más fácil de gestionar (todo en Hostinger)
-  phpMyAdmin disponible para administración visual

¿Necesitas ayuda con la migración? Consulta [DEPLOYMENT.md](#) para instrucciones paso a paso.